ASSIGNMENT-21.3

2303A51303

Shaik Sabina

BATCH-05

Task 1 – AI-Assisted Pair Programming

Prompt: Create a basic Expense Tracker program to record daily expenses (amount, category, date) and display them.

Improve the program by adding:

- Monthly and category-wise summaries
- Input validation
- Menu-driven interface
- Use functions for clean code

Compare both versions and explain improvements briefly.

Code:

```
# Step 1: Initial Version

expenses = []

def add_expense():
    amount = float(input("Enter amount: "))
    category = input("Enter category: ")
    date = input("Enter date (YYYY-MM-DD): ")
    expenses.append({"amount": amount, "category": category, "date": date})
    print("Expense added successfully!")

def display_expenses():
    if not expenses:
        print("No expenses recorded yet.")
        return
    print("\n--- All Recorded Expenses ---")
    for expense in expenses:
        print(f"Amount: {expense['amount']:.2f}, Category: {expense['category']}, Date: {expense['date']}")
    print("-----------------------------")

while True:
    print("\n1. Add Expense")
    print("2. Display Expenses")
    print("3. Exit")
    choice = input("Enter your choice: ")

    if choice == '1':
        add_expense()
    elif choice == '2':
```

```
while True:
    print("\n1. Add Expense")
    print("2. Display Expenses")
    print("3. Exit")
    choice = input("Enter your choice: ")

    if choice == '1':
        add_expense()
    elif choice == '2':
        display_expenses()
    elif choice == '3':
        print("Exiting Expense Tracker. Goodbye!")
        break
    else:
        print("Invalid choice. Please try again.")
```

Output:

```
...
    1. Add Expense
    2. Display Expenses
    3. Exit
    Enter your choice: 1
    Enter amount: 1000
    Enter category: shopping
    Enter date (YYYY-MM-DD): 2026-03-30
    Expense added successfully!

    1. Add Expense
    2. Display Expenses
    3. Exit
    Enter your choice: 2

    --- All Recorded Expenses ---
    Amount: 1000.00, Category: shopping, Date: 2026-03-30
    ---------------------------

    1. Add Expense
    2. Display Expenses
    3. Exit
    Enter your choice: 3
    Exiting Expense Tracker. Goodbye!
```

Explanation: In this task, AI is used as a **pair programmer** to build an Expense Tracker application. First, AI generates a basic program to record and display daily expenses. Then, it improves the program by adding features like summaries, validation, and better code structure.By comparing both versions, we understand how AI helps in **enhancing code quality, adding features, and improving design**, making development faster and more efficient.

Task 2 – Version Control Integration

Prompt: Create a Student Grade Calculator project and manage it using Git.

Steps:

Initialize a Git repository for the project.

Create a new branch and add features:

- Grade classification (A, B, C, etc.)
- Average and percentage calculation

Commit changes with clear and meaningful commit messages.

Merge the feature branch into the main branch and resolve any conflicts.

- Show Git commands used
- Provide sample code changes
- Brief explanation of each step give it in short

Code:

```
def calculate_results(marks):
    total = sum(marks)
    avg = total / len(marks)
    percentage = avg
    if avg >= 90:
        grade = 'A'
    elif avg >= 75:
        grade = 'B'
    elif avg >= 50:
        grade = 'C'
    else:
        grade = 'F'
    return total, avg, percentage, grade
def main():
    marks = []
    n = int(input("Enter number of subjects: "))
    for i in range(n):
        mark = float(input(f"Enter marks for subject {i+1}: "))
        marks.append(mark)
    total, avg, percentage, grade = calculate_results(marks)
    print("\nTotal:", total)
    print("Average:", avg)
    print("Percentage:", percentage)
    print("Grade:", grade)
```

```
def main():
    marks = []
    n = int(input("Enter number of subjects: "))
    for i in range(n):
        mark = float(input(f"Enter marks for subject {i+1}: "))
        marks.append(mark)
    total, avg, percentage, grade = calculate_results(marks)
    print("\nTotal:", total)
    print("Average:", avg)
    print("Percentage:", percentage)
    print("Grade:", grade)
if __name__ == "__main__":
    main()
```

```
git init
git add .
git commit -m "Initial commit: basic grade calculator"

git branch feature-grades
git checkout feature-grades

# after adding features
git add .
git commit -m "Added grade classification and average calculation"

git checkout main
git merge feature-grades
```

Output:

```
PS C:\Users\kamdi\OneDrive\Desktop\AI ASSISTANT COD
ssign-21.py"
Enter number of subjects: 4
Enter marks for subject 1: 56
Enter marks for subject 2: 78
Enter marks for subject 3: 61
Enter marks for subject 4: 96

Total: 291.0
Average: 72.75
Percentage: 72.75
Grade: C
```

Explanation: In this task, Git is used to manage a Student Grade Calculator project. A repository is created, and new features are developed in a separate branch. Changes are committed with proper messages and then merged back into the main branch.This demonstrates how version control helps in **organizing code, tracking changes, and collaborating efficiently** while avoiding conflicts.

Task 3 – AI for Commit Message Generation

Prompt: Generate Git commit messages for a Password Validation module update.

Changes:

- Added minimum length (8 characters)

- Added special character requirement

- Provide short, detailed, and conventional commit messages

- Add one human-written message

- Give a brief comparison of clarity

Code:

```python
import re

def validate_password(password):
    errors = []

    # Rule 1: Minimum length
    if len(password) < 8:
        errors.append("Password must be at least 8 characters long.")

    # Rule 2: At least one special character
    if not re.search(r"[!@#$%^&*(),.?\":{}|<>]", password):
        errors.append("Password must contain at least one special character.")

    if errors:
        return False, errors
    else:
        return True, ["Password is valid"]


# Example usage
password = input("Enter password: ")
is_valid, message = validate_password(password)
```

```python
    # Rule 2: At least one special character
    if not re.search(r"[!@#$%^&*(),.?\":{}|<>]", password):
        errors.append("Password must contain at least one special character.")

    if errors:
        return False, errors
    else:
        return True, ["Password is valid"]


# Example usage
password = input("Enter password: ")
is_valid, message = validate_password(password)

for msg in message:
    print(msg)
```

```
git init
git add password_validation.py
git commit -m "Initial commit: basic password validation module"

# After modifying code (adding rules)
git add password_validation.py
git commit -m "feat: enforce minimum length and special character requirement in

git log
git diff
```

Output:

```
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users
1.py"
Enter password: amul@123
Password is valid
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

Explanation: In this task, the Password Validation module is updated by adding rules like minimum length and special character requirement. AI is used to generate commit messages for these changes.The AI-generated messages are compared with human-written ones. AI messages are more structured and consistent, while human messages are more descriptive.This shows how AI helps in writing clear and professional commit messages.

Task 4 – Pair Programming Simulation

Prompt: Student A: Write a basic quiz program with questions and answers.

Student B: Improve the program by adding:

- Score calculation
- Input validation

Show the updated final code.

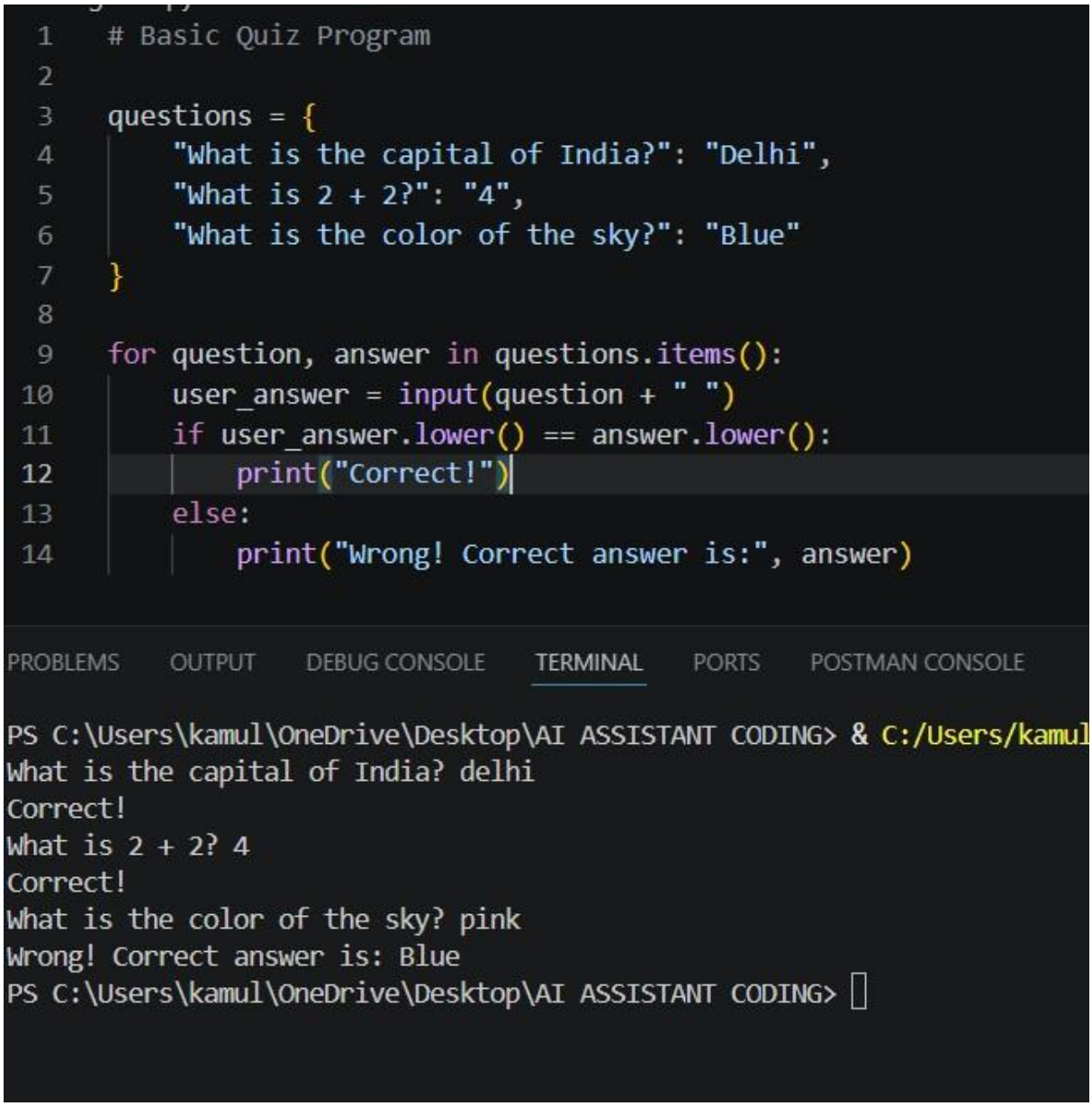
Provide Git commands for branching, merging, and collaboration.

Briefly explain how both students collaborated using Git.

Output:

- Initial Code (Student A)
- Updated Code (Student B)
- Git Commands
- Short Explanation

Intial Code:



Improved code:

```python
# Improved Quiz Program with Score and Input Validation

questions = {
    "What is the capital of India?": "Delhi",
    "What is 2 + 2?": "4",
    "What is the color of the sky?": "Blue"
}

score = 0
total = len(questions)

for question, answer in questions.items():
    while True:
        user_answer = input(question + " ").strip()

        # Input validation (not empty)
        if user_answer == "":
            print("Input cannot be empty. Try again.")
        else:
            break

    if user_answer.lower() == answer.lower():
        print("Correct!")
        score += 1
    else:
        print("Wrong! Correct answer is:", answer)

print(f"\nYour Score: {score}/{total}")
```

Output:

```
What is the capital of India? chennai
Wrong! Correct answer is: Delhi
What is 2 + 2? 4
Correct!
What is the color of the sky? green
Wrong! Correct answer is: Blue

Your Score: 1/3
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

```
# Student A work (main branch)
git add quiz.py
git commit -m "Initial commit: basic quiz program with questions and answers"

# Create branch for Student B
git branch student-b
git checkout student-b

# Student B updates code (score + validation)
git add quiz.py
git commit -m "feat: add score calculation and input validation to quiz system"

# Switch back to main and merge changes
git checkout main
git merge student-b
```

Explanation: In this task, two students collaborate to build a Quiz Management System. Student A creates a basic quiz program, while Student B enhances it by adding score calculation and input validation using AI assistance.They use Git branching and merging to work separately and then combine their changes. This demonstrates how pair programming and version control help in efficient team collaboration and code integration.